

Road vehicles — Diagnostic communication over Controller Area Network (DoCAN) —

基于 CAN 总线实现的 UDS 诊断（DoCAN）

之前写的 UDS 系列的文章介绍的都是应用层的诊断服务，不涉及下层的传输机制，本篇文章简要介绍一下基于 CAN 总线实现的诊断协议的传输层。本文以 classical CAN 为例，CAN FD 原理类似，不单独讲述。

UDS 定义的是诊断服务，属于应用层的内容，实现诊断通信的底层总线技术有很多，比如 CAN，LIN，Ethernet，Flexray 等，由于法规强制的 OBD 接口是 CAN 总线的，所以绝大多数场景中诊断都是基于 CAN 实现的。这就带来了一个问题，classical CAN 总线物理层的每一帧只能传输 8 个字节，CAN FD 第帧最多能传输 64 个字节，那么如果 UDS 产生的一条诊断命令超过了 8 个字节，在 CAN 总线上一帧是承载不了的，就需要进行分包，这也是 DoCAN(Diagnose over CAN)要解决的最主要的问题。

为了实现诊断命令的分包传输，15765-2 总共定义了 4 种类型的帧结构，每种帧结构以数据域的前两个或一个字节来标识（取决于帧类型）。这四种类型分别是：

- 1. SingleFrame
- 2. FirstFrame
- 3. ConsecutiveFrame
- 4. FlowControl

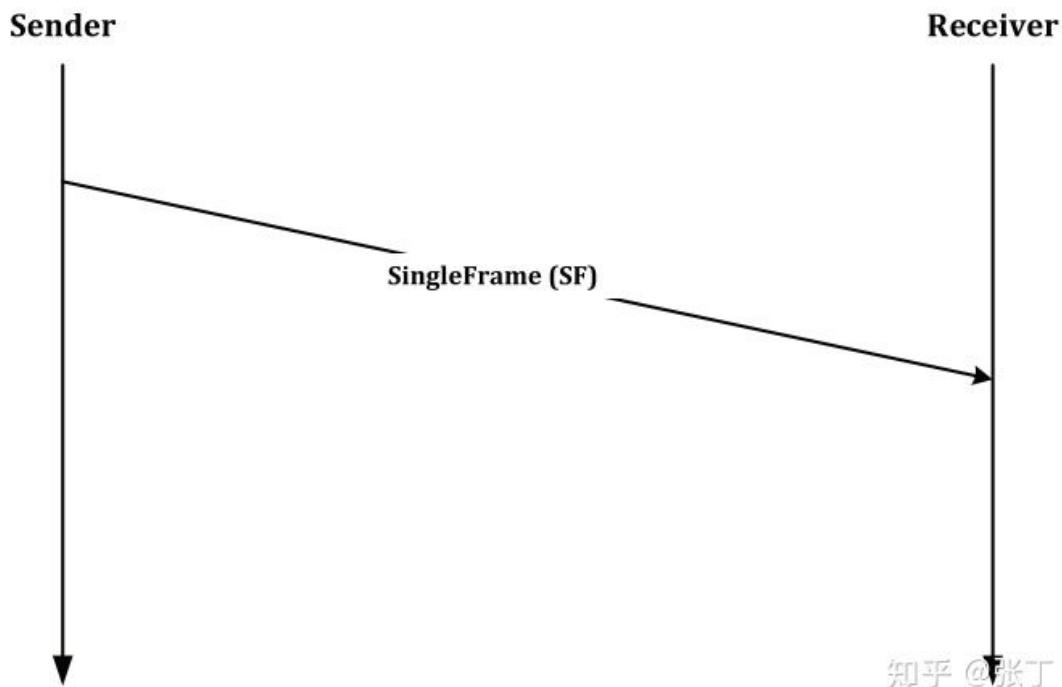
其中 SingleFrame 用于长度不超过 7 个字节的诊断命令或响应。FirstFrame，ConsecutiveFrame，FlowControl 用于传输长度大于 7 个字节的诊断命令或响应。每个诊断帧的第一个字节的高 4bit 用于描述该帧的类型，即该帧属于上述四种中的哪一种。

	byte 1								byte 2	byte 3	byte 4	byte 5	byte 6	byte 7	byte 8
	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0							
SingleFrame	0	0	0	0	SF_DL										
FirstFrame	0	0	0	1	FF_DL										
ConsecutiveFrame	0	0	1	0	SN										
FlowControl	0	0	1	1	FS				BS	Stmin					

传输层的 4 种帧结构

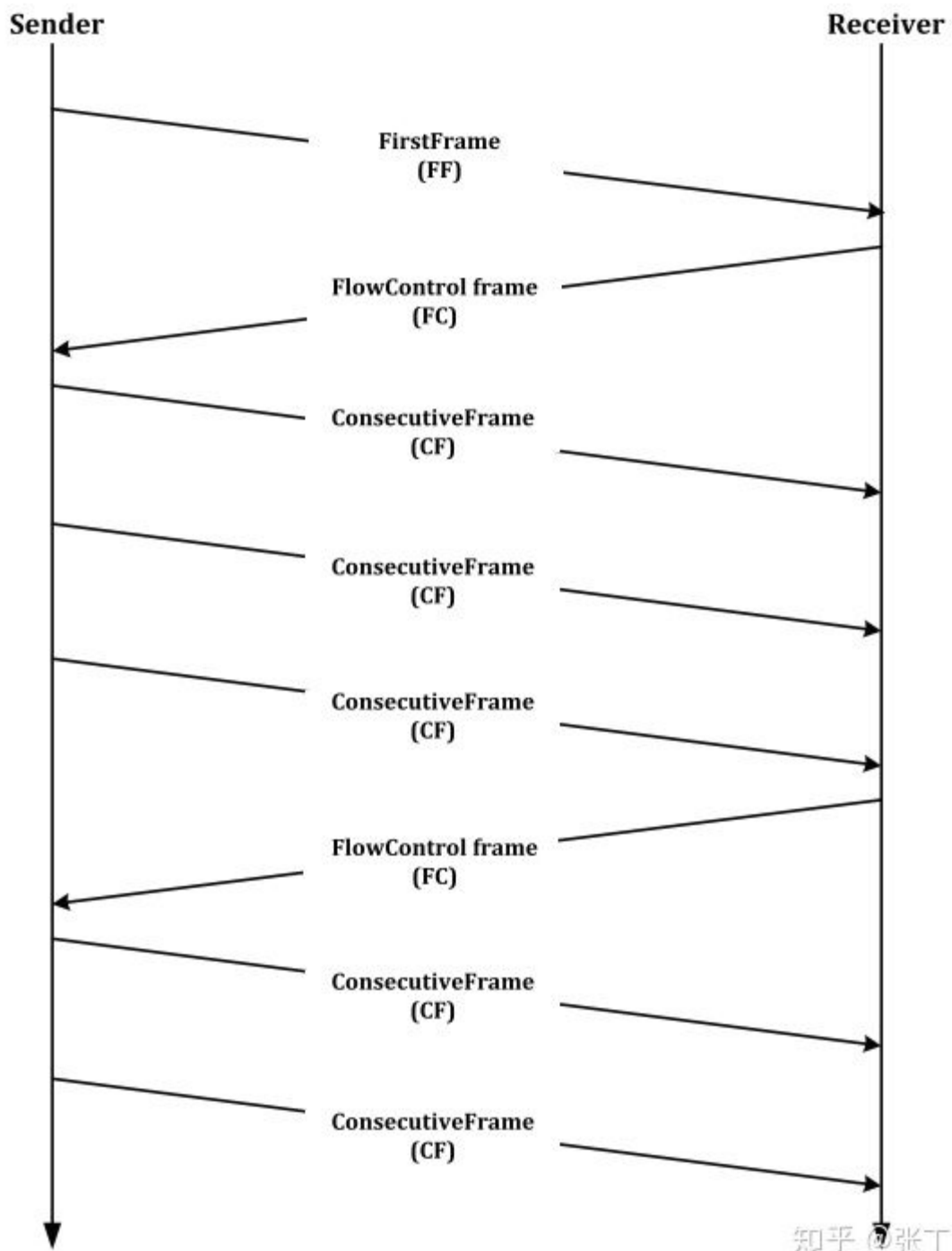
SingleFrame 用于下面这种简单的场景：当诊断报文长度小于等于 7 时，再加上一个字节的 PCI 控制信息就是小于等于 8，可以在一帧 CAN 报文上传输，所以不需要进行分包。此时数据域的

第一个字节高 4bit 值为 0000，标识这是一个帧 SingleFrame，低 4bit 是 SF_DL，即 DataLength，描述后面有几个字节。如果有没有使用的字节，通常要用 0x55 或 0xAA 来填充，因为这两个值的二进制表述其实就是 01010101 和 10101010，这样在 CAN 总线上可以让信号跳变得更频繁一些，不会出现长时间电平不变的情况。



SingleFrame 的应用场景

如果一帧 CAN 报文无法承载一条诊断报文，则需要按照下面的流程进行分包发送：



诊断报文的分段发送

首先，发送端要以一个 FirstFrame 开启通信，告诉接收端还有后续的内容要发，FirstFrame 使用前两个字节作为 PCI 信息，第一个字节高 4bit 为 0001，标识这是一个 FirstFrame，低 4bit 加上第二个字节用于描述总共发送的数据长度是多少（包括在 FirstFrame 中和在 ConsecutiveFrame 中的所有字节数）。

之后接收端发送 FlowControl，告诉发送端能以什么样的速度来发送数据，FlowControl 第一字节的高 4bit 为 0011，低 4bit 为 FS，即 FlowStatus，第二个字节为 BS(BlockSize)，第三个字节为 STmin (SeperateTime)。FlowControl 有 0, 1, 2 三种状态，分别命名为 ContinueToSend (CTS), Wait (WT), Overflow (OVFLW)。如果允许发送端继续发送 ConsecutiveFrame，则 FlowStatus=0；若要求发送端等一会再发送 ConsecutiveFrame，则 FlowStatus=1，再下次允许 sender 发送 ConsecutiveFrame 时，receiver 再发一个 FlowStatus=0 的 FlowControl。如果 receiver 因为资源问题无法接收 sender 发送的数据，则发送一个 FlowStatus=2 的 FlowControl。

BS 指示 sender 一次可以发送多少个 ConsecutiveFrame，当发送 ConsecutiveFrame 数量达到 BS 时，需要 receiver 再次以一个 FlowControl 开启下一波的 ConsecutiveFrame 发送。

receiver 根据自身的接收和处理能力使用 STmin 指示 sender 在发送 ConsecutiveFrame 时最小的时间间隔是多少，从而实现流控制。

ConsecutiveFrame 就是承载 FirstFrame 无法完全承载的剩余数据了，它使用第一个字节用作 PCI，高 4bit 为 0010，低 4bit 用于标识 ConsecutiveFrame 的序列号，从 1 开始，每发送一次 ConsecutiveFrame 增加 1。

不分段传输的诊断服务举例：

request **02** 10 03 55 55 55 55

response **06** 50 03 00 32 01 F4 AA

这是一个请求进入 extended session 的最简单的诊断服务，请求和应答都是 SingleFrame，加粗的 0 标识了 SingleFrame，后面的 2 和 6 表示了这条诊断报文拥有几个字节的数据。

分段传输的诊断服务举例：

这是一个读取 DTC 的命令和应答。

03 19 02 08 55 55 55 55 （诊断仪发送的 SingleFrame 的 request）

10 33 59 02 19 01 00 07 （ECU 以 FirstFrame 开始传输的 response）

30 00 00 55 55 55 55 55 （诊断仪发送的 FlowControl）

21 09 03 05 02 09 05 04 （ECU 发送的 ConsecutiveFrame）

22 07 09 05 06 06 09 05 (ECU 发送的 ConsecutiveFrame)

23 08 03 08 07 01 05 08 (ECU 发送的 ConsecutiveFrame)

24 07 01 06 08 07 01 0C (ECU 发送的 ConsecutiveFrame)

25 08 07 01 0D 08 07 03 (ECU 发送的 ConsecutiveFrame)

26 07 09 08 01 01 09 09 (ECU 发送的 ConsecutiveFrame)

27 01 07 09 AA AA AA AA (ECU 发送的 ConsecutiveFrame, 此时传输结束)

在这里我把所有传输层相关的 PCI 信息都用粗体标识了。

需要提一下的是, BS 和 STmin 等于 0 时, 表示接收端可以以最快的速度来接收数据, 发送端可以一次发送的 ConsecutiveFrame 数量不受限制。